

ITA0612-Machine Learning

CO3 - AT2 - CASE STUDY

Divyesh S P (192424147)

1. Smart Healthcare System using Bayesian Probability Learning

Aim

To detect anomalies in patient vital signs using a Bayesian probability learning model.

Tools Used

- Python
- Google Colab
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

Procedure

1. Generated a synthetic healthcare dataset.
2. Added sensor noise to simulate real-world data.
3. Trained the Gaussian Naïve Bayes model.
4. Predicted patient health conditions.
5. Evaluated the model using accuracy and confusion matrix.

Code

```
# =====
# SMART HEALTHCARE SYSTEM USING BAYESIAN PROBABILITY LEARNING
# =====

# Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# -----
# Step 1 : Generate Synthetic Healthcare Dataset
# -----

np.random.seed(42)

samples = 1000

heart_rate = np.random.randint(55, 130, samples)
temperature = np.round(np.random.normal(98.6, 2, samples), 1)
oxygen = np.random.randint(85, 100, samples)
blood_pressure = np.random.randint(90, 170, samples)

condition = []

for hr, temp, spo2, bp in zip(heart_rate, temperature, oxygen,
blood_pressure):

    if (
        hr > 110 or
        hr < 60 or
        temp > 101 or
        temp < 97 or
        spo2 < 92 or
        bp > 145 or
        bp < 95
    ):
        condition.append("Anomaly")
    else:
        condition.append("Normal")

# Create DataFrame
data = pd.DataFrame({
    "HeartRate": heart_rate,
    "Temperature": temperature,
    "Oxygen": oxygen,
    "BloodPressure": blood_pressure,
    "Condition": condition
})
```

```

print("First Five Records")
print(data.head())

print("\nDataset Shape:", data.shape)

# -----
# Step 2 : Introduce Sensor Noise
# -----

noise = np.random.normal(0, 1, samples)

data["HeartRate"] += noise.astype(int)

# -----
# Step 3 : Prepare Data
# -----

X = data[["HeartRate",
          "Temperature",
          "Oxygen",
          "BloodPressure"]]

y = data["Condition"]

# -----
# Step 4 : Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

# -----
# Step 5 : Train Bayesian Model
# -----

model = GaussianNB()

model.fit(X_train, y_train)

# -----
# Step 6 : Prediction
# -----

y_pred = model.predict(X_test)

# -----
# Step 7 : Accuracy
# -----

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy")

```

```

print(round(accuracy*100,2), "%")

# -----
# Step 8 : Classification Report
# -----

print("\nClassification Report\n")

print(classification_report(y_test, y_pred))

# -----
# Step 9 : Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=["Anomaly", "Normal"],
    yticklabels=["Anomaly", "Normal"]
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()

# -----
# Step 10 : Prediction on New Patient
# -----

new_patient = pd.DataFrame({

    "HeartRate": [118],
    "Temperature": [102],
    "Oxygen": [89],
    "BloodPressure": [155]

})

prediction = model.predict(new_patient)

print("\nNew Patient Data")

print(new_patient)

print("\nPredicted Health Status:", prediction[0])

# -----
# Step 11 : Feature Distribution

```

```
# -----  
  
plt.figure(figsize=(8,5))  
  
plt.hist(data["HeartRate"], bins=20)  
  
plt.title("Heart Rate Distribution")  
  
plt.xlabel("Heart Rate")  
  
plt.ylabel("Frequency")  
  
plt.show()  
  
# -----  
# Step 12 : Oxygen Level Distribution  
# -----  
  
plt.figure(figsize=(8,5))  
  
plt.hist(data["Oxygen"], bins=15)  
  
plt.title("Oxygen Saturation Distribution")  
  
plt.xlabel("SpO2")  
  
plt.ylabel("Frequency")  
  
plt.show()  
  
# -----  
# Step 13 : Temperature Distribution  
# -----  
  
plt.figure(figsize=(8,5))  
  
plt.hist(data["Temperature"], bins=20)  
  
plt.title("Body Temperature Distribution")  
  
plt.xlabel("Temperature")  
  
plt.ylabel("Frequency")  
  
plt.show()  
  
# -----  
# Step 14 : Blood Pressure Distribution  
# -----  
  
plt.figure(figsize=(8,5))  
  
plt.hist(data["BloodPressure"], bins=20)  
  
plt.title("Blood Pressure Distribution")  
  
plt.xlabel("Blood Pressure")
```

```
plt.ylabel("Frequency")

plt.show()

# -----
# Step 15 : Dataset Summary
# -----

print("\nDataset Summary\n")

print(data.describe())

print("\nProgram Executed Successfully.")
```

Output

First Five Records

	HeartRate	Temperature	Oxygen	BloodPressure	Condition
0	106	98.1	98	134	Normal
1	69	96.4	98	123	Anomaly
2	126	103.7	89	98	Anomaly
3	115	98.7	87	97	Anomaly
4	75	98.6	97	96	Normal

Dataset Shape: (1000, 5)

Accuracy
90.4 %

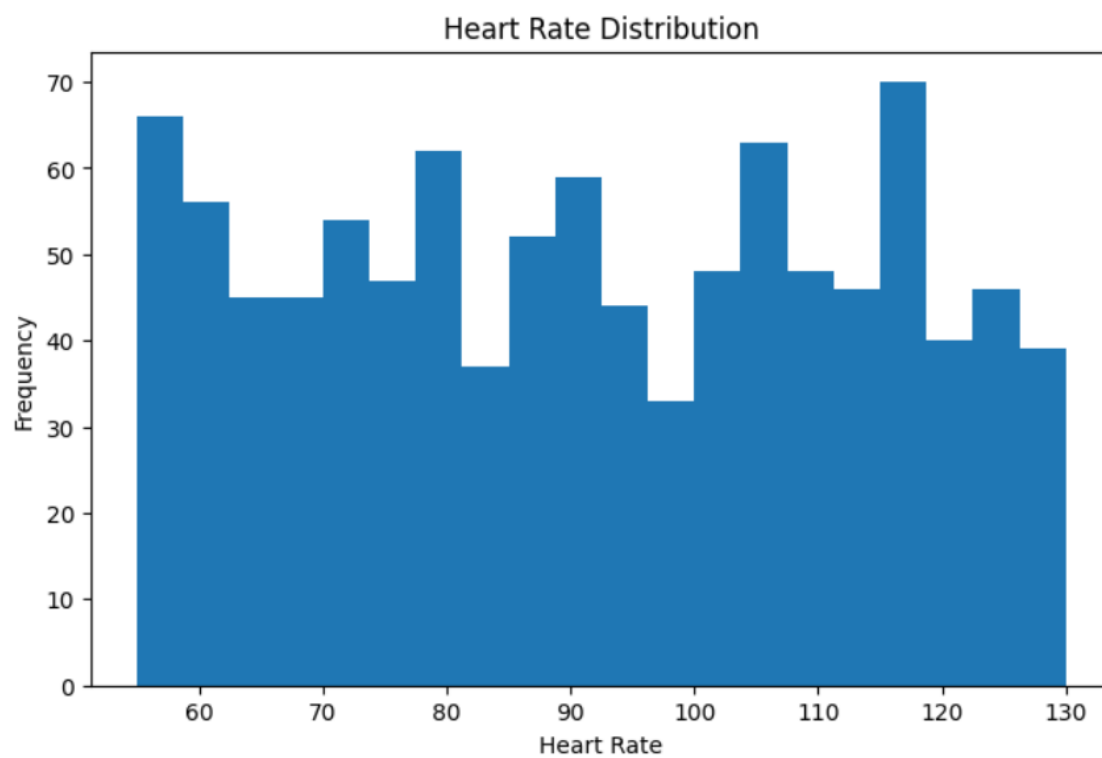
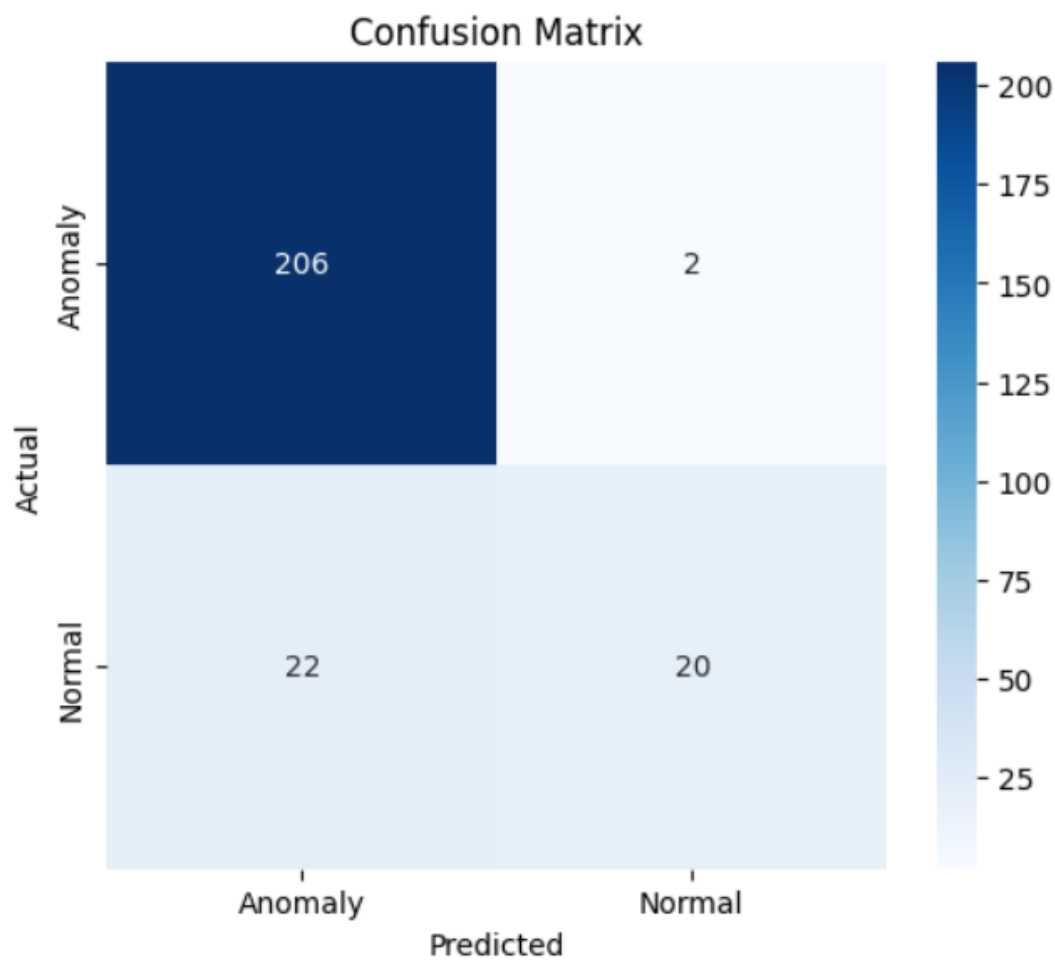
Classification Report

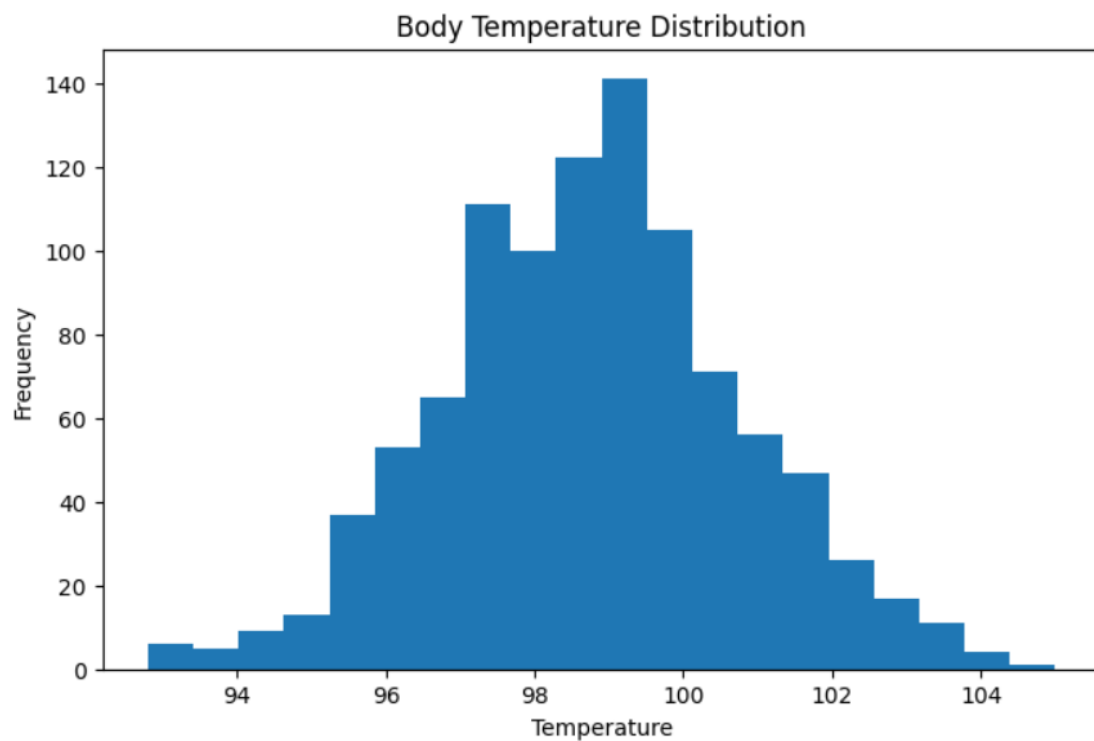
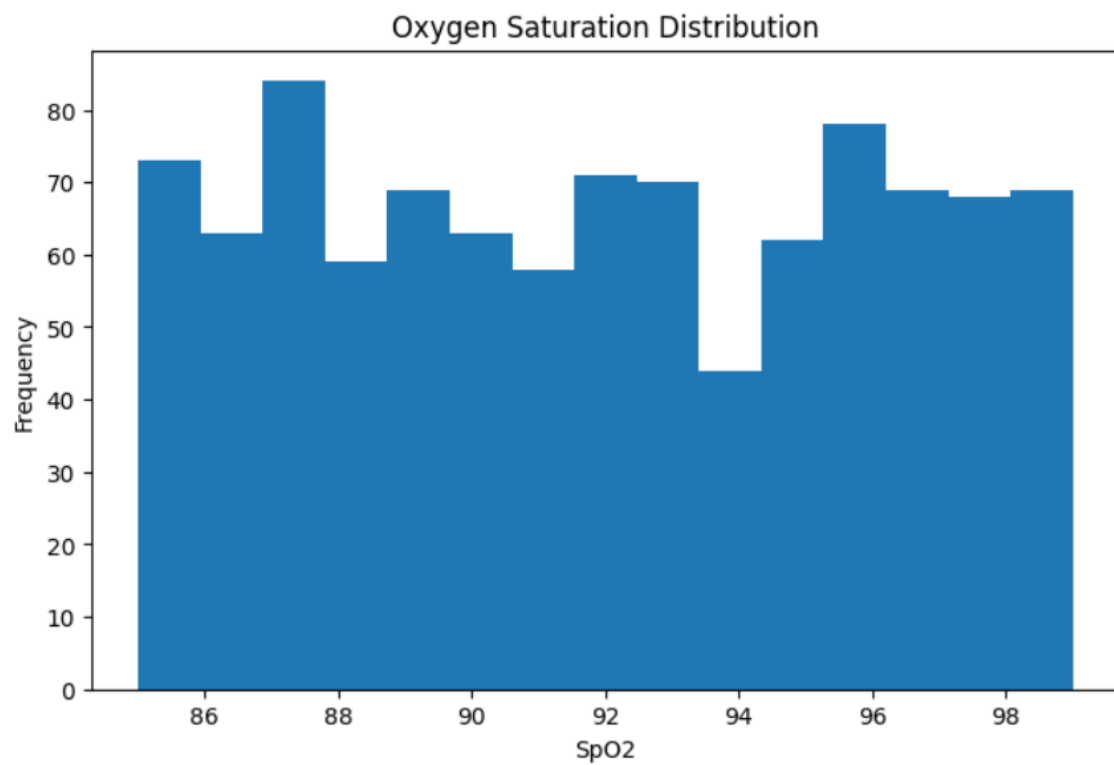
	precision	recall	f1-score	support
Anomaly	0.90	0.99	0.94	208
Normal	0.91	0.48	0.62	42
accuracy			0.90	250
macro avg	0.91	0.73	0.78	250
weighted avg	0.90	0.90	0.89	250

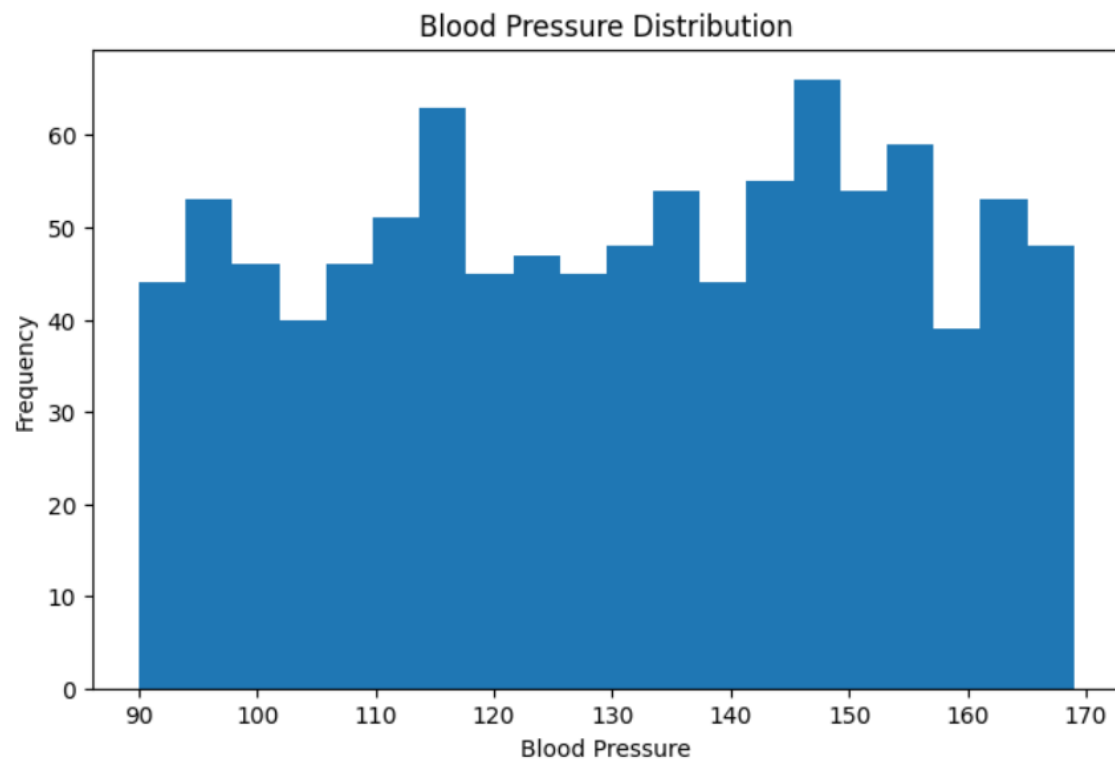
New Patient Data

	HeartRate	Temperature	Oxygen	BloodPressure
0	118	102	89	155

Predicted Health Status: Anomaly







Dataset Summary

	HeartRate	Temperature	Oxygen	BloodPressure
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	91.394000	98.760300	91.956000	130.234000
std	21.844953	1.998607	4.409479	22.844893
min	55.000000	92.800000	85.000000	90.000000
25%	72.000000	97.400000	88.000000	111.000000
50%	91.000000	98.700000	92.000000	131.000000
75%	111.000000	100.000000	96.000000	150.000000
max	130.000000	105.000000	99.000000	169.000000

Program Executed Successfully.

Result

The Bayesian model successfully detected normal and abnormal patient conditions

2. Cybersecurity Intrusion Detection using Bayesian Learning

Aim

To detect network intrusions using Naïve Bayes and the EM algorithm.

Tools Used

- Python
- Google Colab
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

Procedure

1. Generated a synthetic network dataset.
2. Added and handled missing values.
3. Trained the Gaussian Naïve Bayes classifier.
4. Applied the EM (Gaussian Mixture) algorithm.
5. Evaluated the model performance.

Code

```
# =====  
# CYBERSECURITY INTRUSION DETECTION SYSTEM  
# USING NAÏVE BAYES + EM (GAUSSIAN MIXTURE)  
# =====  
  
# Import Libraries  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder
```

```

from sklearn.naive_bayes import GaussianNB
from sklearn.mixture import GaussianMixture
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# -----
# Step 1 : Generate Synthetic Dataset
# -----

np.random.seed(42)

samples = 1000

packets = np.random.randint(100, 6000, samples)
duration = np.random.randint(1, 300, samples)
failed_logins = np.random.randint(0, 10, samples)
bytes_sent = np.random.randint(500, 50000, samples)

protocol = np.random.choice(
    ["TCP", "UDP", "ICMP"],
    samples
)

attack = []

for p, d, f, b in zip(
    packets,
    duration,
    failed_logins,
    bytes_sent):

    if (
        p > 4000 or
        f >= 5 or
        d > 220 or
        b > 40000
    ):
        attack.append("Attack")
    else:
        attack.append("Normal")

# Create DataFrame

data = pd.DataFrame({

    "Packets": packets,
    "Duration": duration,
    "FailedLogins": failed_logins,
    "BytesSent": bytes_sent,
    "Protocol": protocol,
    "Traffic": attack

})

print("First Five Records\n")
print(data.head())

print("\nDataset Shape :", data.shape)

```

```

# -----
# Step 2 : Introduce Missing Values
# -----

for col in ["Packets", "Duration"]:

    idx = np.random.choice(
        data.index,
        20,
        replace=False
    )

    data.loc[idx, col] = np.nan

# Handle Missing Values

data.fillna(data.mean(numeric_only=True), inplace=True)

# -----
# Step 3 : Encode Protocol
# -----

encoder = LabelEncoder()

data["Protocol"] = encoder.fit_transform(data["Protocol"])

# -----
# Step 4 : Prepare Features
# -----

X = data.drop("Traffic", axis=1)

y = data["Traffic"]

# -----
# Step 5 : Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(

    X,
    y,
    test_size=0.25,
    random_state=42

)

# -----
# Step 6 : Naïve Bayes Model
# -----

nb = GaussianNB()

nb.fit(X_train, y_train)

y_pred = nb.predict(X_test)

```

```

# -----
# Step 7 : Evaluation
# -----

accuracy = accuracy_score(y_test, y_pred)

print("\nNaïve Bayes Accuracy")

print(round(accuracy * 100, 2), "%")

print("\nClassification Report\n")

print(classification_report(y_test, y_pred))

# -----
# Step 8 : Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(

    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Attack", "Normal"],
    yticklabels=["Attack", "Normal"]

)

plt.title("Naïve Bayes Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()

# -----
# Step 9 : EM Algorithm (Gaussian Mixture)
# -----

gmm = GaussianMixture(

    n_components=2,
    random_state=42

)

gmm.fit(X)

clusters = gmm.predict(X)

data["Cluster"] = clusters

```

```

print("\nFirst Ten Cluster Assignments\n")

print(data[["Traffic", "Cluster"]].head(10))

# -----
# Step 10 : Cluster Visualization
# -----

plt.figure(figsize=(8,6))

plt.scatter(

    data["Packets"],
    data["BytesSent"],
    c=data["Cluster"]

)

plt.xlabel("Packets")

plt.ylabel("Bytes Sent")

plt.title("EM Algorithm Clustering")

plt.show()

# -----
# Step 11 : Protocol Distribution
# -----

plt.figure(figsize=(6,4))

sns.countplot(x=data["Protocol"])

plt.title("Protocol Distribution")

plt.xlabel("Protocol")

plt.ylabel("Count")

plt.xticks([0,1,2], ["ICMP", "TCP", "UDP"])

plt.show()

# -----
# Step 12 : Traffic Distribution
# -----

plt.figure(figsize=(6,4))

sns.countplot(x=data["Traffic"])

plt.title("Traffic Type")

plt.show()

# -----
# Step 13 : Predict New Network Activity

```

```

# -----

new_network = pd.DataFrame({

    "Packets": [5200],
    "Duration": [250],
    "FailedLogins": [8],
    "BytesSent": [45000],
    "Protocol": [encoder.transform(["TCP"])[0]]

})

prediction = nb.predict(new_network)

print("\nNew Network Activity\n")

print(new_network)

print("\nPrediction :", prediction[0])

# -----
# Step 14 : Dataset Summary
# -----

print("\nDataset Summary\n")

print(data.describe())

print("\nProgram Executed Successfully.")

```

Output

First Five Records

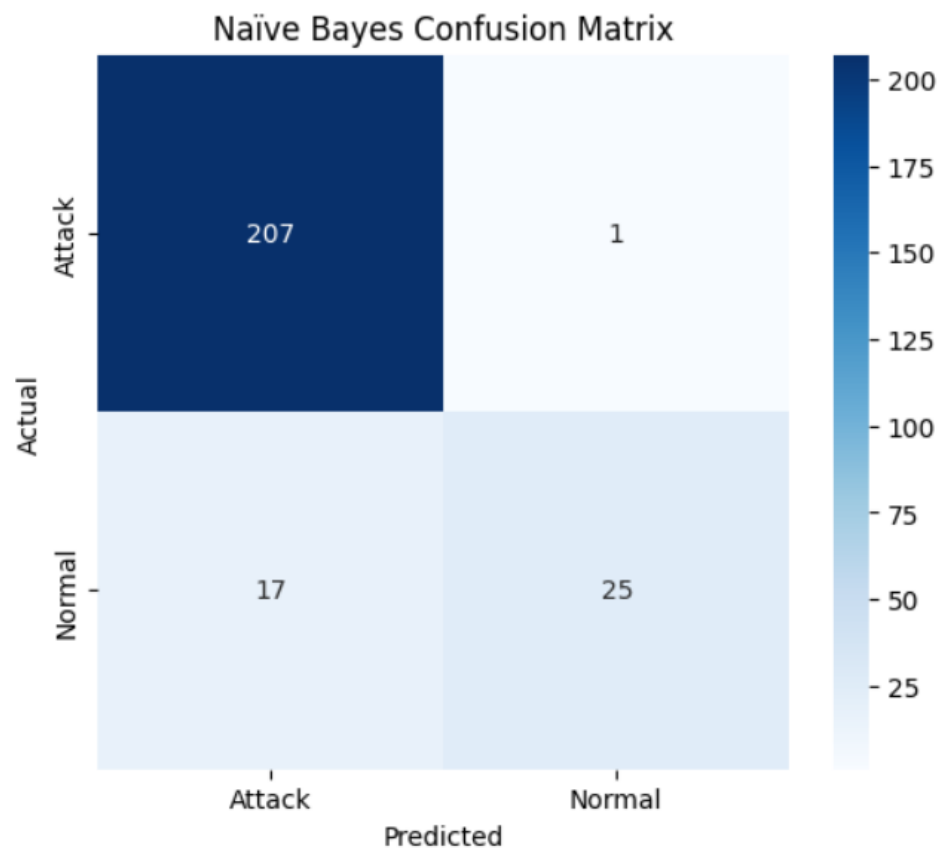
	Packets	Duration	FailedLogins	BytesSent	Protocol	Traffic
0	960	225	1	20039	ICMP	Attack
1	5490	20	0	12277	UDP	Attack
2	5326	113	6	34896	ICMP	Attack
3	5291	140	4	30873	ICMP	Attack
4	3872	47	0	33654	ICMP	Normal

Dataset Shape : (1000, 6)

Naïve Bayes Accuracy
92.8 %

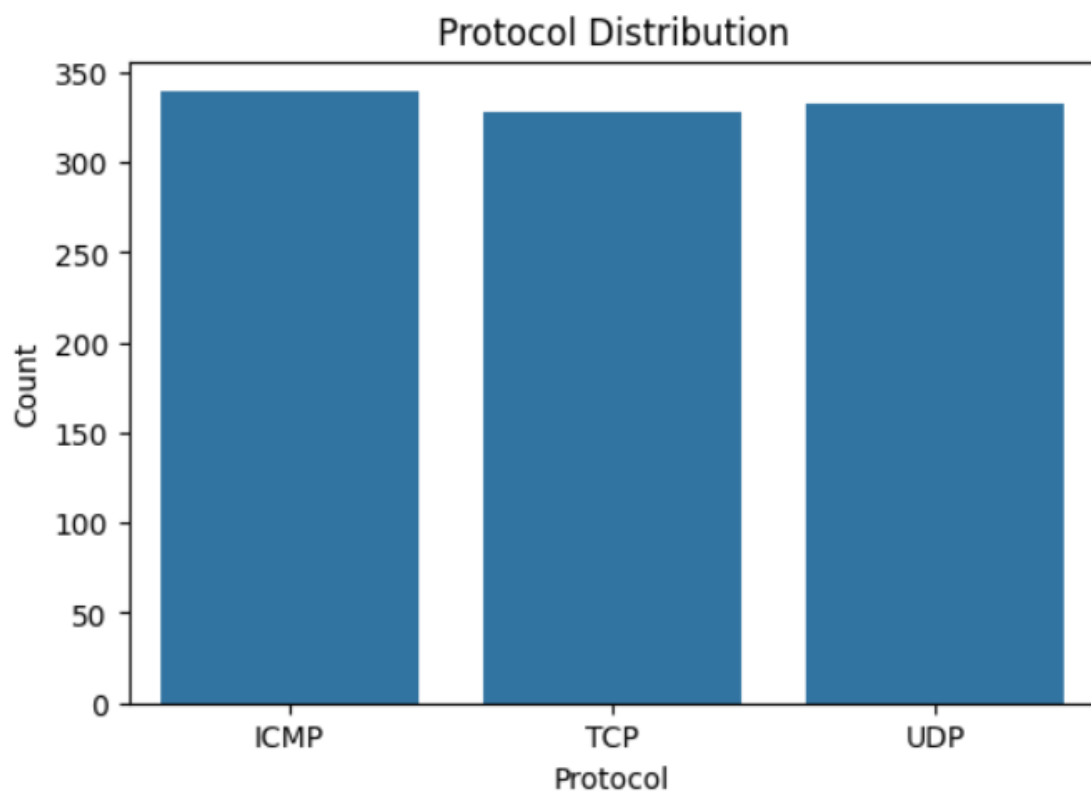
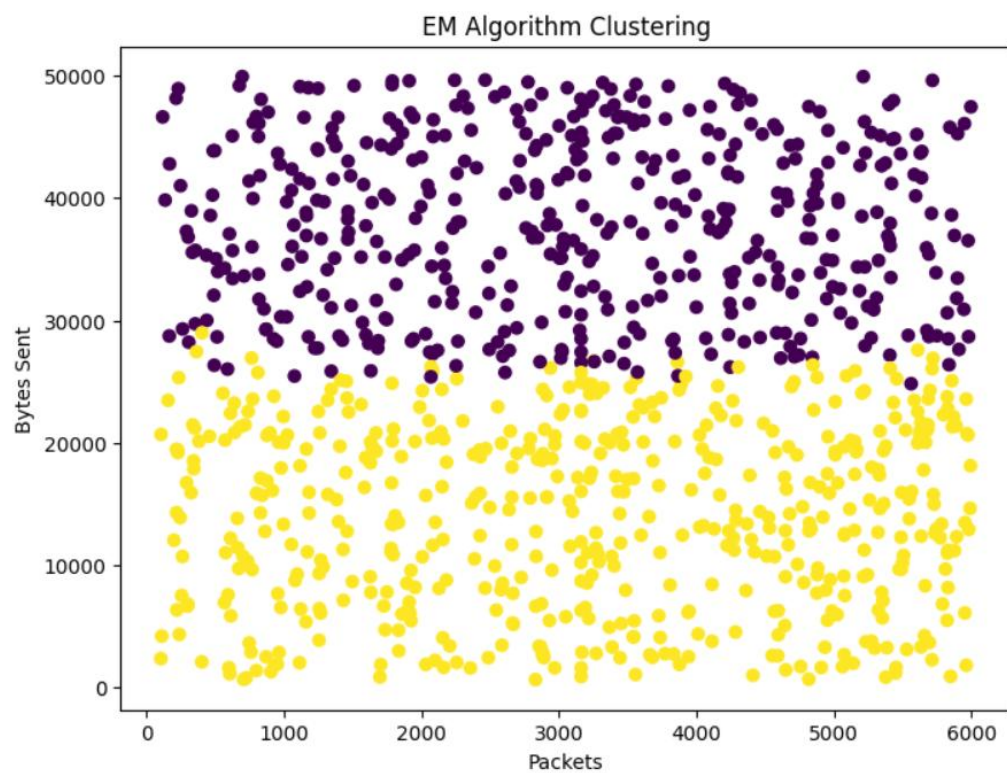
Classification Report

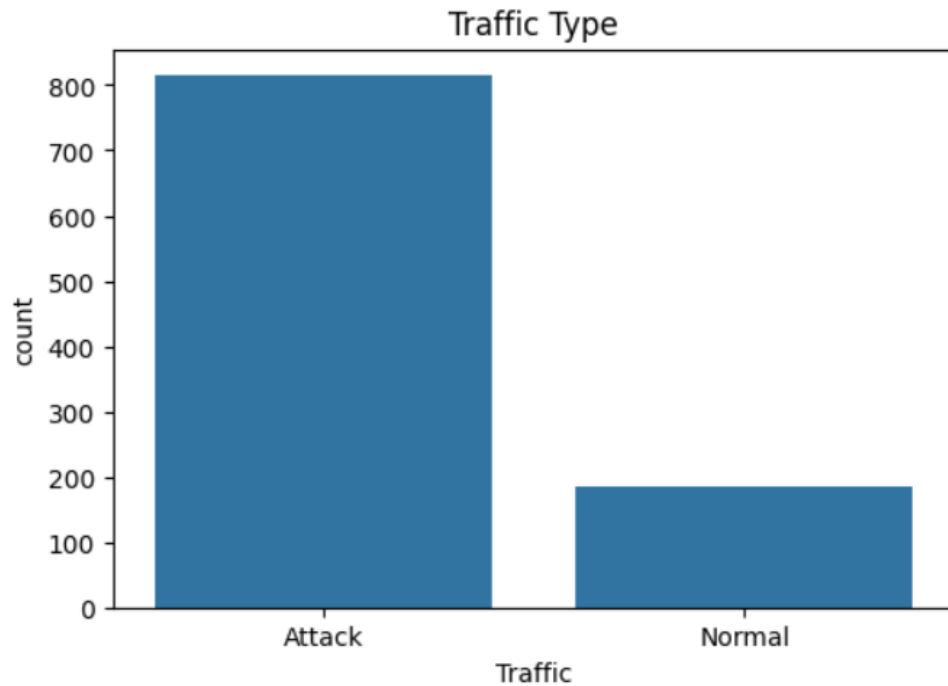
	precision	recall	f1-score	support
Attack	0.92	1.00	0.96	208
Normal	0.96	0.60	0.74	42
accuracy			0.93	250
macro avg	0.94	0.80	0.85	250
weighted avg	0.93	0.93	0.92	250



First Ten Cluster Assignments

	Traffic	Cluster
0	Attack	1
1	Attack	1
2	Attack	0
3	Attack	0
4	Normal	0
5	Attack	1
6	Attack	0
7	Normal	1
8	Attack	1
9	Attack	1





New Network Activity

Packets	Duration	FailedLogins	BytesSent	Protocol	
0	5200	250	8	45000	1

Prediction : Attack

Dataset Summary

	Packets	Duration	FailedLogins	BytesSent	Protocol	\
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	
mean	3161.368367	148.052041	4.46100	25023.568000	0.994000	
std	1677.978428	87.576629	2.81183	13988.001163	0.820144	
min	103.000000	1.000000	0.00000	638.000000	0.000000	
25%	1759.250000	75.000000	2.00000	13191.250000	0.000000	
50%	3161.368367	148.052041	4.00000	24614.500000	1.000000	
75%	4648.750000	223.000000	7.00000	36943.500000	2.000000	
max	5998.000000	299.000000	9.00000	49930.000000	2.000000	

	Cluster
count	1000.000000
mean	0.534000
std	0.499092
min	0.000000
25%	0.000000
50%	1.000000
75%	1.000000
max	1.000000

Program Executed Successfully.

Result

The system successfully identified normal and attack traffic with good accuracy.

3. Crop Yield Prediction using Bayesian Inference

Aim

To predict crop yield using Bayesian inference based on environmental conditions.

Tools Used

- Python
- Google Colab
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

Procedure

1. Generated a synthetic crop dataset.
2. Encoded categorical values.
3. Trained the Gaussian Naïve Bayes model.
4. Predicted crop yield.
5. Evaluated the model using accuracy and confusion matrix.

Code

```
# =====  
# CROP YIELD PREDICTION USING BAYESIAN INFERENCE  
# =====  
  
# Import Libraries  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder  
from sklearn.naive_bayes import GaussianNB
```

```

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# -----
# Step 1 : Generate Synthetic Dataset
# -----

np.random.seed(42)

samples = 1000

rainfall = np.random.randint(50, 300, samples)      # mm
temperature = np.random.randint(18, 42, samples)    # °C
humidity = np.random.randint(40, 95, samples)       # %
soil_quality = np.random.choice(
    ["Poor", "Average", "Good"],
    samples,
    p=[0.2, 0.4, 0.4]    # Prior agricultural knowledge
)

yield_level = []

for r, t, h, s in zip(rainfall, temperature, humidity, soil_quality):
    if (
        120 <= r <= 220 and
        22 <= t <= 32 and
        h >= 60 and
        s == "Good"
    ):
        yield_level.append("High")

    elif (
        r < 80 or
        t > 37 or
        s == "Poor"
    ):
        yield_level.append("Low")

    else:
        yield_level.append("Medium")

# -----
# Step 2 : Create DataFrame
# -----

data = pd.DataFrame({
    "Rainfall": rainfall,
    "Temperature": temperature,
    "Humidity": humidity,
    "SoilQuality": soil_quality,
    "CropYield": yield_level
})

print("First Five Records\n")
print(data.head())

```

```

print("\nDataset Shape :", data.shape)

# -----
# Step 3 : Encode Soil Quality
# -----

encoder = LabelEncoder()

data["SoilQuality"] = encoder.fit_transform(data["SoilQuality"])

# -----
# Step 4 : Prepare Features
# -----

X = data.drop("CropYield", axis=1)
y = data["CropYield"]

# -----
# Step 5 : Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

# -----
# Step 6 : Train Bayesian Model
# -----

model = GaussianNB()

model.fit(X_train, y_train)

# -----
# Step 7 : Prediction
# -----

y_pred = model.predict(X_test)

# -----
# Step 8 : Accuracy
# -----

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy")

print(round(accuracy * 100, 2), "%")

# -----
# Step 9 : Classification Report

```

```

# -----

print("\nClassification Report\n")

print(classification_report(y_test, y_pred))

# -----
# Step 10 : Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(

    cm,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=model.classes_,
    yticklabels=model.classes_

)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()

# -----
# Step 11 : Predict New Farm
# -----

new_farm = pd.DataFrame({

    "Rainfall": [180],
    "Temperature": [27],
    "Humidity": [72],
    "SoilQuality": [encoder.transform(["Good"])[0]]

})

prediction = model.predict(new_farm)

print("\nNew Farm Conditions\n")

print(new_farm)

print("\nPredicted Crop Yield :", prediction[0])

# -----
# Step 12 : Rainfall Distribution
# -----

plt.figure(figsize=(7,5))

```

```

plt.hist(data["Rainfall"], bins=20)

plt.title("Rainfall Distribution")

plt.xlabel("Rainfall (mm)")

plt.ylabel("Frequency")

plt.show()

# -----
# Step 13 : Temperature Distribution
# -----

plt.figure(figsize=(7,5))

plt.hist(data["Temperature"], bins=15)

plt.title("Temperature Distribution")

plt.xlabel("Temperature (°C)")

plt.ylabel("Frequency")

plt.show()

# -----
# Step 14 : Crop Yield Distribution
# -----

plt.figure(figsize=(6,4))

sns.countplot(x=data["CropYield"])

plt.title("Crop Yield Classes")

plt.show()

# -----
# Step 15 : Correlation Heatmap
# -----

numeric_data = data.copy()

numeric_data["CropYield"] =
LabelEncoder().fit_transform(data["CropYield"])

plt.figure(figsize=(7,6))

sns.heatmap(

    numeric_data.corr(),
    annot=True,
    cmap="YlGnBu"

)

plt.title("Feature Correlation")

```

```
plt.show()

# -----
# Step 16 : Dataset Summary
# -----

print("\nDataset Summary\n")

print(data.describe())

print("\nProgram Executed Successfully.")
```

Output

First Five Records

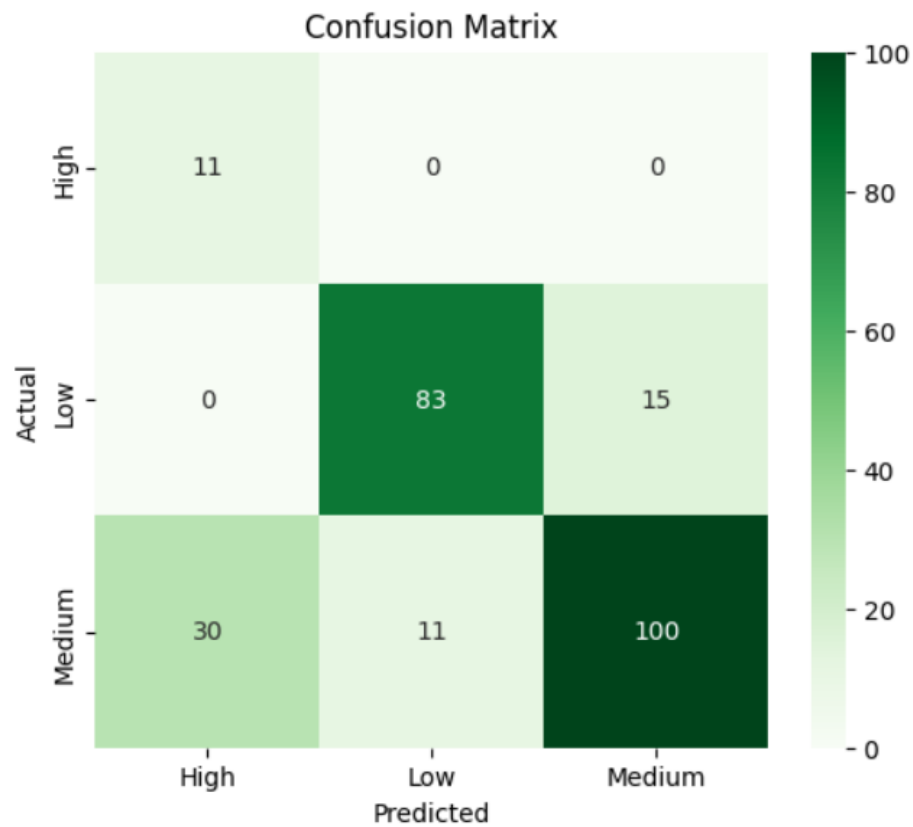
	Rainfall	Temperature	Humidity	SoilQuality	CropYield
0	152	32	43	Average	Medium
1	229	34	59	Poor	Low
2	142	28	71	Average	Medium
3	64	38	75	Average	Low
4	156	39	89	Good	Low

Dataset Shape : (1000, 5)

Model Accuracy
77.6 %

Classification Report

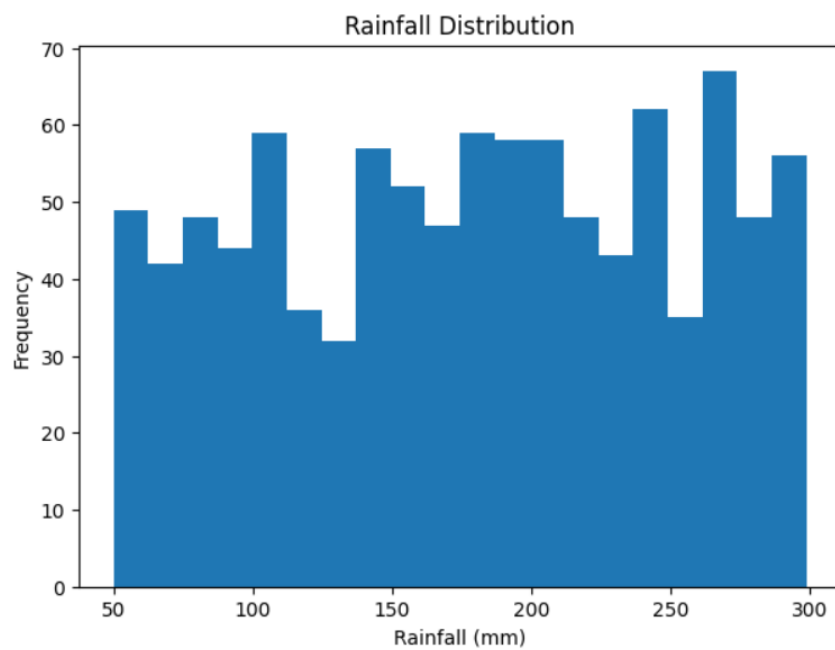
	precision	recall	f1-score	support
High	0.27	1.00	0.42	11
Low	0.88	0.85	0.86	98
Medium	0.87	0.71	0.78	141
accuracy			0.78	250
macro avg	0.67	0.85	0.69	250
weighted avg	0.85	0.78	0.80	250

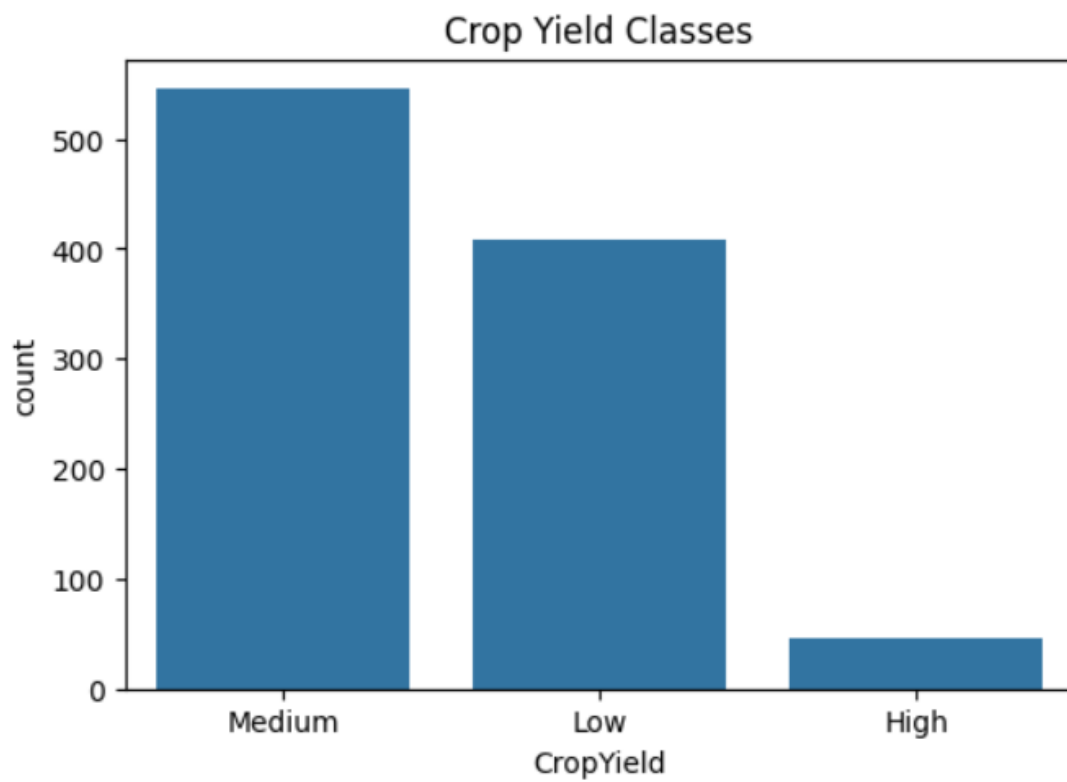
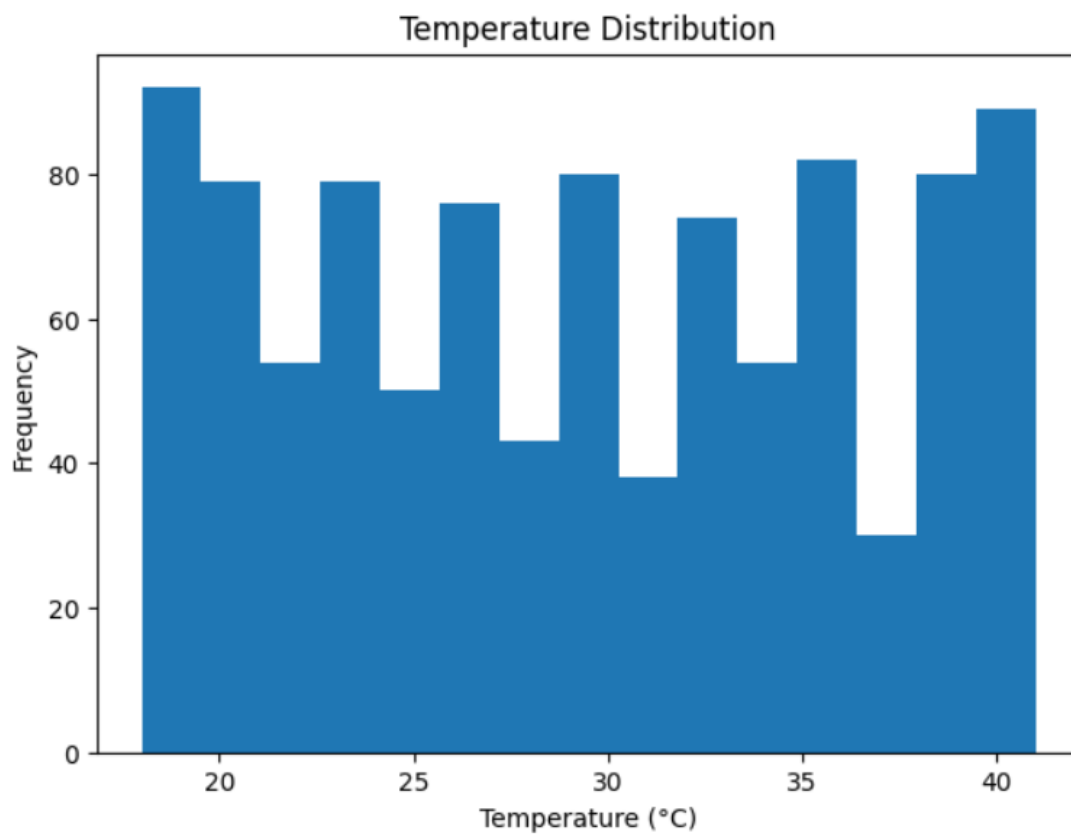


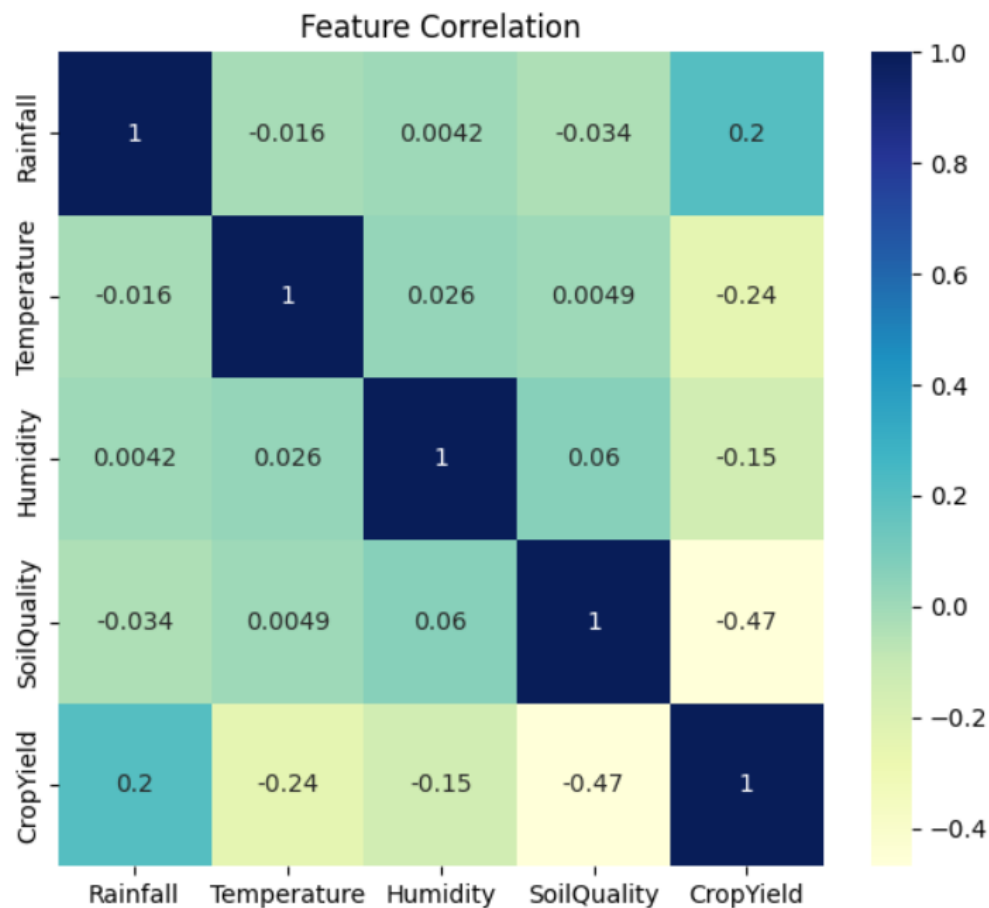
New Farm Conditions

	Rainfall	Temperature	Humidity	SoilQuality
0	180	27	72	1

Predicted Crop Yield : High







Dataset Summary

	Rainfall	Temperature	Humidity	SoilQuality
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	178.500000	29.31800	66.209000	0.800000
std	71.610111	7.00828	15.857136	0.744684
min	50.000000	18.00000	40.000000	0.000000
25%	115.750000	23.00000	53.000000	0.000000
50%	180.000000	29.00000	65.500000	1.000000
75%	239.000000	35.00000	80.000000	1.000000
max	299.000000	41.00000	94.000000	2.000000

Program Executed Successfully.

Result

The Bayesian model successfully predicted crop yield with high accuracy.